Practical 8: KNN Classification

```
# Practical 8 - Code 1: KNN with Scaling, Stratified Split & Metrics

import numpy as np
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_s

# Load dataset
data = load_iris()
X = data.data
y = data.target

# Stratified train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Feature scaling
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Train KNN
knn = KNeighborsClassifier(n_neighbors=5, metric='euclidean')
knn.fit(X_train_scaled, y_train)

# Predictions
y_pred = knn.predict(X_test_scaled)

# Evaluation
print("Accuracy :", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred, average='weighted'))
print("Recall   :", recall_score(y_test, y_pred, average='weighted'))
print("F1 Score :", f1_score(y_test, y_pred, average='weighted'))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

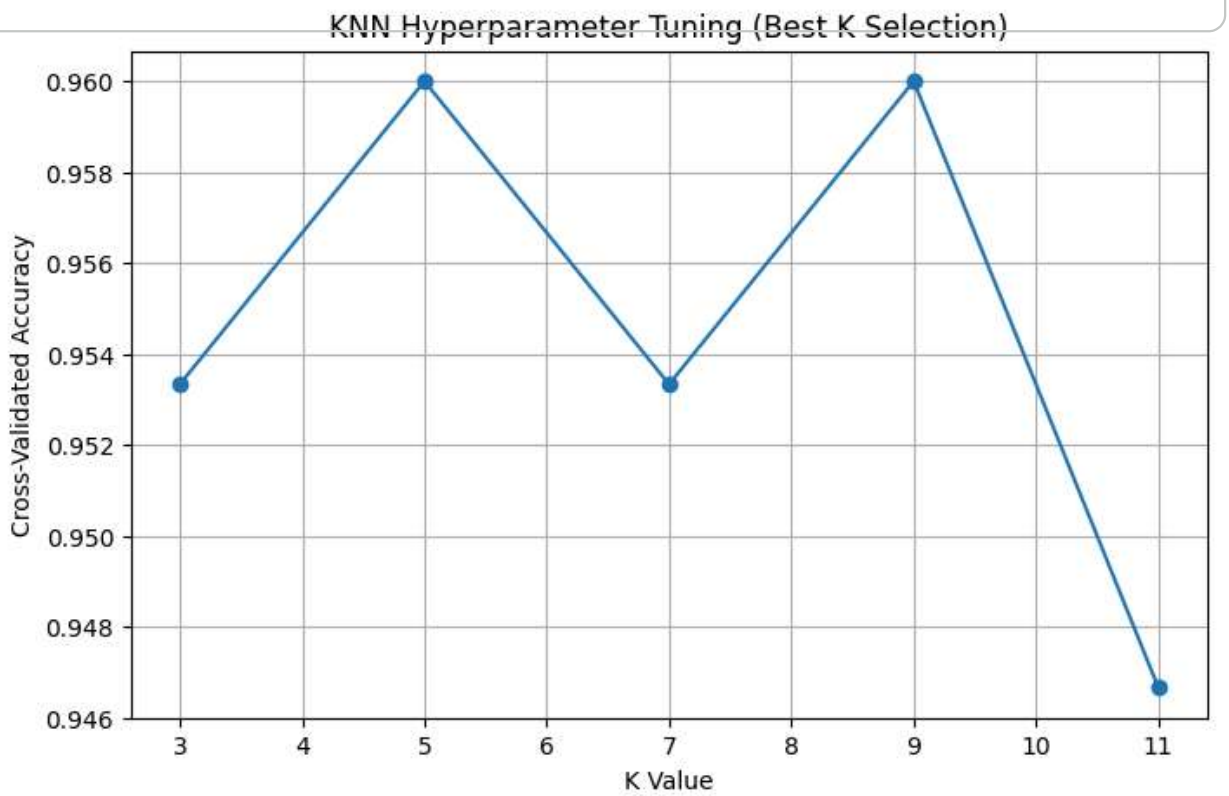
```
Accuracy : 0.9333333333333333
Precision: 0.9444444444444445
Recall   : 0.9333333333333333
F1 Score : 0.9326599326599326
```

```
Confusion Matrix:
[[10  0  0]
 [ 0 10  0]
 [ 0  2  8]]
```

```
# Practical 8 - Code 2: Distance Metric Comparison

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# Load dataset
X, y = load_iris(return_X_y=True)

# Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Scale features
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Different distance metrics
metrics = ['euclidean', 'manhattan', 'minkowski']

for m in metrics:
    model = KNeighborsClassifier(n_neighbors=5, metric=m)
    model.fit(X_train, y_train)
    preds = model.predict(X_test)
    acc = accuracy_score(y_test, preds)
    print(f"Metric: {m} | Accuracy: {acc:.4f}")
```

```
Metric: euclidean | Accuracy: 0.9333
Metric: manhattan | Accuracy: 0.9333
Metric: minkowski | Accuracy: 0.9333
```

```
# Practical 8 - Code 3: Hyperparameter Tuning with Cross-Validation

import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier

# Load data
X, y = load_iris(return_X_y=True)
```

```
# Scale features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Try multiple K values
k_values = [3, 5, 7, 9, 11]
cv_scores = []

for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k)
    scores = cross_val_score(knn, X_scaled, y, cv=5, scoring='accuracy')
    cv_scores.append(scores.mean())
    print(f"K = {k}, CV Accuracy = {scores.mean():.4f}")

# Best K
best_k = k_values[np.argmax(cv_scores)]
print(f"\nBest K found: {best_k}")

# Plot results
plt.figure(figsize=(8, 5))
plt.plot(k_values, cv_scores, marker='o')
plt.xlabel("K Value")
plt.ylabel("Cross-Validated Accuracy")
plt.title("KNN Hyperparameter Tuning (Best K Selection)")
plt.grid(True)
plt.show()
```

```
K = 3, CV Accuracy = 0.9533
K = 5, CV Accuracy = 0.9600
K = 7, CV Accuracy = 0.9533
K = 9, CV Accuracy = 0.9600
K = 11, CV Accuracy = 0.9467

Best K found: 5
```